4. Git-HOL

Git Merge Conflict Resolution

What is a Merge Conflict?

A **merge conflict** happens when two branches modify the **same file** in different ways, and Git cannot decide which version to keep.

Git pauses the merge and asks you to resolve the conflict manually.

Objective

Learn how to:

- Create branches
- Make changes in different branches
- Create a merge conflict
- Resolve the conflict
- Complete the merge successfully

Step 1: Check if the master branch is clean

git status

Output

On branch master

nothing to commit, working tree clean

This means there are no pending changes.

Step 2: Create a new branch

git checkout -b GitWork

or

git switch -c GitWork

Now you are working in the **GitWork** branch.

Step 3: Create hello.xml

touch hello.xml

Add content:

<message>Hello from GitWork Branch</message>

Check status

git status

Git shows

Untracked files:

hello.xml

Step 4: Commit the file

git add hello.xml

git commit -m "Added hello.xml in GitWork branch"

Now the file is saved in the GitWork branch.

Step 5: Switch back to master

git checkout master

Step 6: Create the same file in master

Create

hello.xml

Add different content

<message>Hello from Master Branch</message>

Notice that both branches have a file with the **same name**, but the contents are different.

Step 7: Commit the changes

git add hello.xml

git commit -m "Added hello.xml in master"

Step 8: View Git history

git log --oneline --graph --decorate --all

Example

* a7f2c1 Added hello.xml in GitWork

|\

| * d31fa8 Added hello.xml in master

|/

This graph shows where the branches split.

Step 9: Compare both branches

git diff master GitWork

Git displays the differences between the two branches.

Step 10: View differences using P4Merge

If P4Merge is installed:

git difftool

P4Merge provides a graphical comparison of the files.

Step 11: Merge GitWork into master

Switch to master if necessary.

git checkout master

Merge

git merge GitWork

Git now reports a conflict.

Example

Auto-merging hello.xml

CONFLICT (content): Merge conflict in hello.xml

Automatic merge failed.

Step 12: Observe Git conflict markers

Open

hello.xml

You will see

<<<<<<< HEAD

<message>Hello from Master Branch</message>

=======

<message>Hello from GitWork Branch</message>

>>>>>>> GitWork

Meaning:

- <<<<<<< HEAD → Current branch (master)
- ======= → Separator
- >>>>>>> GitWork → Incoming changes from GitWork

Step 13: Resolve the conflict

Edit the file manually.

Example:

<message>Hello from Master Branch</message>

<message>Hello from GitWork Branch</message>

or choose whichever version you want.

Remove all the conflict markers.

Save the file.

Step 14: Complete the merge

Stage the resolved file.

git add hello.xml

Commit

git commit -m "Resolved merge conflict in hello.xml"

The merge is now complete.

Step 15: Ignore backup files

Sometimes merge tools create backup files like

hello.xml.orig

Open

.gitignore

Add

*.orig

Now Git ignores backup files.

Commit

git add .gitignore

git commit -m "Added backup files to .gitignore"

Step 16: View all branches

git branch

Example

* master

GitWork

Step 17: Delete the merged branch

git branch -d GitWork

Output

Deleted branch GitWork

Step 18: View the final history

git log --oneline --graph --decorate

Example

```
*   98ab321 Resolved merge conflict
|\
| * 7c1234 Added hello.xml in GitWork
* | 3b4567 Added hello.xml in master
|/
* 1a2b3c Initial commit
```

The graph shows both branches joined together after the merge.

Common Git Commands Used

Command	Purpose
git status	Check repository status
git checkout -b GitWork	Create and switch to a new branch
git checkout master	Switch to the master branch
git add <file>	Stage changes
git commit -m "message"	Save changes
git merge GitWork	Merge the GitWork branch into master
git diff master GitWork	Compare two branches
git log --oneline --graph --decorate --all	Show branch history graph
git branch	List all branches

Command	Purpose
git branch -d GitWork	Delete a merged branch